

1. Remove Processes Master List Update Status, Master List Update Month, Master List Source Sheet and remove columns from Dashboard, Templates
2. Something made Banners, CP Due Date and Unlocked CP reappear during processing,
3. After all processing and secondary rows are removed, reduce the sheet to 1000 rows.
4. On Disenrolled List - Hide rows older than 12 months.
5. Enforce Global Order put Template - Disenrolled Exclusion next to Disenrolled Exclusion
Also Unhid everything and did not rehide anything.
6. Create Menu trigger for one step System Set up - to Run these processes
"rebuildFormatDashboardDefaults", "setupSystemSheets",
"createIndexSheet", "runDashboardQualityStartUp",
7. Create Menu Trigger for a one step Build All Templates - Run
"createOrRefreshAllReportTemplates", "runDashboardQualityValidateTemplates"

Updated Menu:

Master List

Data & Processing Engine

-  Format Monthly Sheets
-  Demo P
 -  Update Demo P
 -  Build Demo P
-  Create / Update Disenrolled List
-  Create Master List

Sheet & Layout Management

-  Enforce Global Tab Sorting Order
-  Monthly Import Sheets
 -  Hide Monthly Import Sheets
 -  Archive Monthly Import Sheets
-  Monthly Active Sheets
 -  Hide Monthly Active Sheets
 -  Archive Monthly Active Sheets
-  Templates
 - Hide Templates
 - Show Templates
-  System Sheets
 - Hide System Sheets
 - Show System Sheets
-  Clear Timing & Quality Logs
-  Framework Timing on/off

Quick Start-up

-  System Set up - Run "rebuildFormatDashboardDefaults", "setupSystemSheets", "createIndexSheet", "runDashboardQualityStartUp",
-  Build All Templates - Run "createOrRefreshAllReportTemplates", "runDashboardQualityValidateTemplates"
-  Dashboard Quality Workflow

Maintenance/Rebuild

-  Quality
 - Dashboard Quality Start up
 - Dashboard Quality Validate Templates
 - Dashboard Quality Workflow
-  Format Sheets
 - Banner
 - CP Due Date
 - Unlocked CP
 - Raw Data
-  Start - up
 -  Set up System Sheets
 -  Format Dashboard
 -  Save Active Layout as Rebuild Default
 -  Create / Refresh All Templates
 -  Build Index

Legacy / "Ghost" Sheets (Targeted for Auto-Deletion)

The script still contains references to the following sheet names inside its cleanup arrays (SYSTEM_SHEETS_TO_HIDE and deleteLegacyOperationalAndDiagnosticSheets_). If the script detects these in an older workbook, it will **automatically delete or hide them**:

- Archive Index & Archive Index Settings
- Framework Test 6 Date Format Report
- Framework Test 9 Monthly Change Subheader Report
- Framework Test 10 Dashboard Audit Report
- Framework Health Check Report
- Timing Summary Report
- Master List Health Check
- Care Plan Sync Diagnostics
- Monthly Change Validation
- Format Demo P Timing, Update Master List Timing, Create Master List Timing

The Master List Audit Purge

We can safely rip these massive audit sheet generators out of the script. Since the underlying data merge logic is still used quietly in the background to physically move the rows into the Master List, we will just delete the user-facing audit builders.

Pass this prompt to your engineering assistant to purge the dead audit code:

Act as an expert Google Apps Script engineer.

Our architecture has evolved. All data conflict resolution, contact flattening, and primary row assignment now strictly occurs upstream between Raw Data and Demo P. Therefore, the standalone Master List Merge Audits are obsolete.

Please optimize the codebase by completely deleting the following 3 functions, which remove over 300 lines of dead audit-generation code:

1. Delete ``runUpdateMasterListMergeAudit()``
2. Delete ``buildUpdateMasterListMergeAuditForMonth_()``
3. Delete ``buildUpdateMasterListPrimaryRowAuditForMonth_()``

Do not delete the smaller helper functions (like ``getPrimaryMergeRowItem_`` or ``getMergeAuditChangedFields_``), as those are still utilized by the background ``updateMasterListFromMonthlyChangeActions_`` engine to safely move the resolved data.

This is a brilliant architectural catch. You have identified a genuine "blind spot" in the sync loop.

The "MCR Blind Spot" Explained

Currently, the **Monthly Change Report** (MCR) only scans for changes across specific, hardcoded arrays of columns (e.g., `RAW_DEMO_P_DEMOGRAPHIC_FIELDS`, `RAW_DEMO_P_CONTACT_FIELDS`, etc.).

If a piece of data changes in **Raw Data** that is **not** included in those specific MCR tracking arrays (like a newly added custom column or a minor untracked data point), the MCR ignores it. Because your **Demo P** sync strictly relies on `getDemoPMonthlySyncChangedPMRs_` to decide what needs updating, it will never pull that fresh data over, causing **Demo P** to silently fall out of sync with **Raw Data**.

The Solution: The Cryptographic Hash Safety Net

You already have the perfect tool built into the framework to fix this: **Source Hashes**.

Whenever **Raw Data** is processed, it generates an MD5 hash (`computeStableHash_`) that represents the exact state of that participant's entire row block. We can add a simple safety net to the `getDemoPMonthlySyncChangedPMRs_` function.

Instead of *only* trusting the MCR, it will also compare the **Source Hash** of the incoming **Raw Data** against the **Source Hash** currently sitting on **Demo P**. If the hashes don't match, it proves the data has mutated, and it will forcefully queue that PMR for a sync—even if the MCR completely missed it.

The Patch

Pass this exact prompt to your engineering assistant to inject the Hash Drift Safety Net into your sync engine:

Act as an expert Google Apps Script engineer.

We need to add a cryptographic safety net to our Demo P monthly sync. Currently, 'Demo P' relies entirely on the Monthly Change Report to identify changed PMRs. If a column changes that the MCR isn't tracking, Demo P misses the update.

Please fully replace the 'getDemoPMonthlySyncChangedPMRs_' function with this upgraded version. It adds a secondary "Hash Drift" check that compares the Raw Data MD5 hash to the Demo P MD5 hash, forcing an update if any data mutation is detected:

```
```javascript
```

```
function getDemoPMonthlySyncChangedPMRs_(monthParts) {
```

```
 const ss = SpreadsheetApp.getActiveSpreadsheet();
```

```
 const set = new Set();
```

```
 // 1. STANDARD MCR PASS: Collect explicitly flagged PMRs from the Monthly Change Report
```

```
 const mcrSheet = getMonthlySheetByPrefixAndDate_(ss, MONTHLY_CHANGE_REPORT_PREFIX,
 monthParts.firstDay, monthParts.lastDay);
```

```
 if (mcrSheet) {
```

```
 const data = getDataValues_(mcrSheet, HEADER_ROW, DATA_START_ROW);
```

```
const pmrIdx = getPMRIndex_(data.headerMap);
```

```
if (pmrIdx !== -1) {
```

```
 data.values.forEach(function(row) {
```

```
 const pmr = normalizePMR_(row[pmrIdx]);
```

```
 if (pmr) set.add(pmr);
```

```
 });
```

```
}
```

```
}
```

```
// 2. CRYPTOGRAPHIC HASH DRIFT SAFETY NET: Catch untracked columns or MCR misses
```

```
const demoSheet = getCurrentDemoPSheet_(monthParts);
```

```
const rawSheet = getCurrentRawDataSheet_(monthParts);
```

```
if (demoSheet && rawSheet) {
```

```
 const demoData = getDataValues_(demoSheet, HEADER_ROW, DATA_START_ROW);
```

```
 const rawData = getDataValues_(rawSheet, HEADER_ROW, DATA_START_ROW);
```

```
 const demoPmrIdx = getPMRIndex_(demoData.headerMap);
```

```
 const demoHashIdx = demoData.headerMap["Source Hash"];
```

```
 const rawPmrIdx = getPMRIndex_(rawData.headerMap);
```

```
 if (demoPmrIdx !== -1 && demoHashIdx !== undefined && rawPmrIdx !== -1) {
```

```
 // Build a map of what the hashes SHOULD be based on fresh Raw Data
```

```
 const rawHashesByPmr = buildSourceHashByPMR_(rawData, rawPmrIdx);
```

```
 const demoPmrSet = new Set();
```

```

demoData.values.forEach(function(row) {

 const pmr = normalizePMR_(row[demoPmrIdx]);

 const demoHash = String(row[demoHashIdx] || "").trim();

 if (pmr) {

 demoPmrSet.add(pmr);

 // Force update if the Raw Data hash differs from the saved Demo P hash
 if (rawHashesByPmr[pmr] && rawHashesByPmr[pmr] !== demoHash) {

 set.add(pmr);

 }

 }

});

// Catch brand new PMRs that exist in Raw Data but haven't made it to Demo P yet
rawData.values.forEach(function(row) {

 const pmr = normalizePMR_(row[rawPmrIdx]);

 if (pmr && !demoPmrSet.has(pmr)) {

 set.add(pmr);

 }

});

}

}

return set;

```

}

## The "Other Changes" Patch

Pass this exact prompt to your engineering assistant to inject this dynamic catch-all section into your framework:

Act as an expert Google Apps Script engineer.

We are future-proofing our Monthly Change Report (MCR) to ensure absolute 1-to-1 visibility with Raw Data. Currently, if a user adds a new column to Raw Data, the MCR ignores it because it isn't in our hardcoded tracking arrays (Demographic, Caseload, Contact, Banner).

We need to add a dynamic "Other Changes" section that acts as a catch-all for any column not explicitly tracked in the existing arrays.

Please make the following surgical updates to the Monthly Change Report engine:

1. In 'getMonthlyChangeSectionSpecs\_', add the new "Other Changes" section to the bottom of the returned array:

```
```javascript
{
  title: "Other Changes",
  pmrSet: sectionData.otherPMRs,
  rowMode: "dobOnly",
  changedColumnsByPMR: sectionData.otherChangedColumnsByPMR
}
```

2. Update the 'RFF_MONTHLY_CHANGE_SUBSECTIONS' global constant array to include "Other Changes" at the end.
3. Update 'compareRawDemoPForSectionReport_'.

- Create a combined `Set` of all currently tracked fields (Demographic + Contact + Caseload + Banner + Enrollment + Disenrollment).
 - Dynamically generate an `OTHER_FIELDS` array by filtering `currentData.headers`. If a header is not in the tracked fields `Set` (and isn't a system column like "Primary PMR Row"), push it to `OTHER_FIELDS`.
 - Add `const otherPMRs = new Set();` and `const otherChangedColumnsByPMR = new Map();`.
 - Inside the main participant comparison loop, run the `getChangedColumnsForSectionRows_` check using the new `OTHER_FIELDS` array. If changes are found, add them to `otherPMRs` and `otherChangedColumnsByPMR`.
 - Return `otherPMRs` and `otherChangedColumnsByPMR` in the final output object.
4. Update `'findNextMonthlyChangeSectionTitleRow_'` and `'getChangedPMRsFromMonthlyChangeReport_'` so their internal `titles / sectionTitles` arrays include "Other Changes" so the system knows how to read the new block.

Act as an elite Google Apps Script Principal Systems Engineer.

I am providing a monolithic Google Apps Script framework that has flawless data integrity but is hitting Google's 6-minute execution timeout limits. I need you to implement three specific architectural optimizations to drastically reduce execution overhead.

Please make the following 3 surgical updates to the codebase:

1. DECOUPLE THE EXTERNAL ARCHIVE INDEX (Reduces 75+ seconds of overhead)

Currently, ``createIndexSheet()`` calls ``SpreadsheetApp.openById(RFF_ARCHIVE_SPREADSHEET_ID)`` every time it runs to map cold-storage Drive files. Opening external workbooks synchronously causes massive lag.

- Modify ``createIndexSheet()`` to entirely remove the ``SpreadsheetApp.openById`` call and the "External Drive Cold-Storage Archives" right-side matrix logic. The ``finalIndexMatrix`` should be reduced to only output Columns A-D (the local Active Operational Sheets Workspace).

- Create a new, standalone function called ``refreshCloudArchiveIndex()`` that contains the extracted ``SpreadsheetApp.openById`` logic. This new function should build and format a dedicated "Archive Index" sheet so users can fetch cloud records only when explicitly requested.

- Update the ``ML_MENU_CALLBACKS`` and ``onOpen`` UI menu to include a button for "Refresh Cloud Archive Index" pointing to this new function.

2. THROTTLE DASHBOARD QUALITY WRITES (Reduces ~39 seconds of overhead)

The framework is wasting time painting the UI of the Quality Report at the end of every data pass.

- In the CONFIGURATION section, locate ``let RFF_DASHBOARD_QUALITY_DEFER_WRITES_ = false;`` and change it to ``let RFF_DASHBOARD_QUALITY_DEFER_WRITES_ = true;``. This permanently defers UI rendering of the quality sheet during background data pipelines.

3. DISABLE MASSIVE FORMATTING EXTENSIONS (Reduces UI paint lag on large data sets)

Copying format rules down 6,500+ rows is crashing the Sheets UI renderer.

- In the CONFIGURATION section, locate ``const RFF_OUTPUT_EXTEND_TEMPLATE_FORMATTING = true;`` and change it to ``const RFF_OUTPUT_EXTEND_TEMPLATE_FORMATTING = false;``.

Provide the updated code blocks for the CONFIGURATION section, the MENU FUNCTIONS section, and the modified ``createIndexSheet()`` / new ``refreshCloudArchiveIndex()`` functions. Do not rewrite the entire 16,000-line script; just output the sections and functions that require modification.